23CSE455 **Design Patterns** — Case Study Report

Design Pattern Identification & Evolution

in a Spring Boot Legal Case Management Application

Team B11
Application `vikash0439/legalSystem` (commit 4e4ebb57)
Date 2026-10-10
Repository 23CSE455_DP_CaseStudy

Roll number	Student name	Batch
Am.sc.u4cse23237	Lokesh	CSE
Am.sc.u4cse23249	Sai Tharun	CSE
Am.sc.u4cse23268	Rakshann	CSE
Am.sc.u4cse23311	Nithilesh	CSE

At a glance

Confirmed pattern types (baseline)	**5**
Recorded instances (incl. 1 idiom, 1 uncertain)	**7**
Changes implemented (CH01–CH03) + refactor	**3 + 1**
Cross-language implementations (4 patterns × 4 langs)	**16**
Part 2 unit tests (`mvn test`)	**7 pass / 0 fail**
Application boots with changes (MySQL)	**yes**

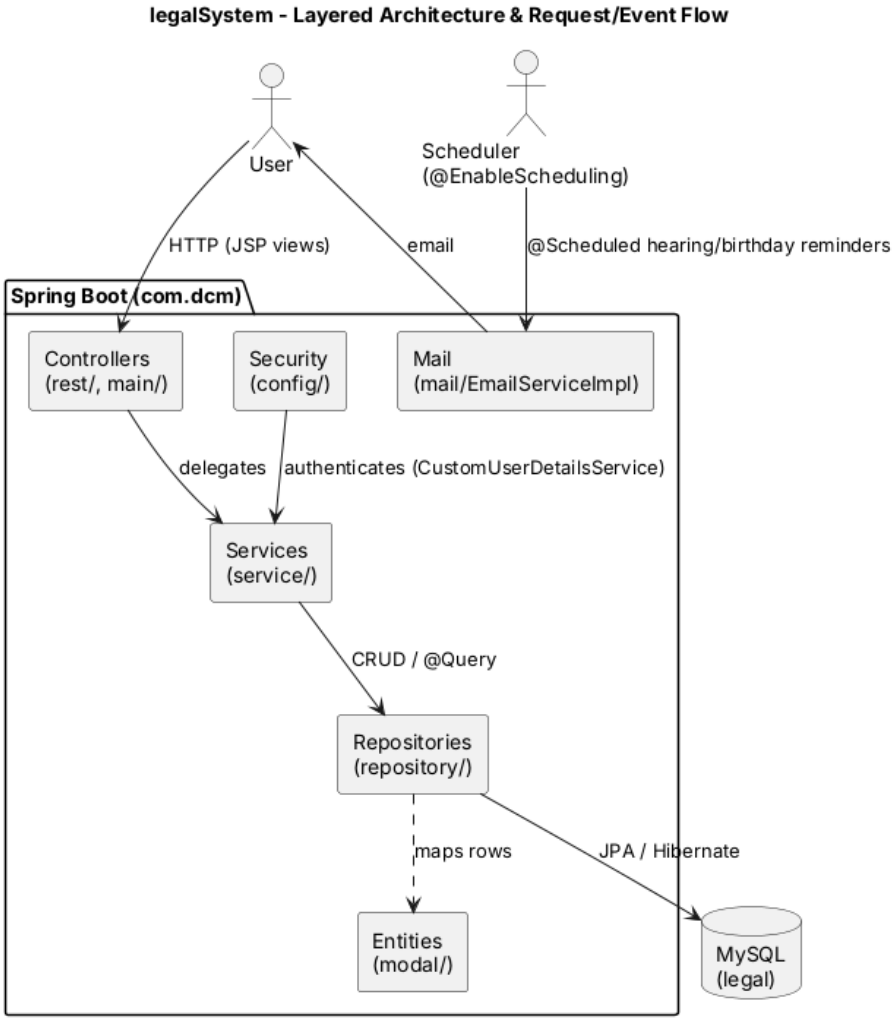
1 Application and architecture

Original source: https://github.com/vikash0439/legalSystem; baseline commit `4e4ebb574d6730614855899` (2023-03-18); accessed 2026-10-10. **License:** the repository contains no LICENSE file; the code is therefore used for non-commercial academic study only, under the course's fair-use scope, with the author credited.

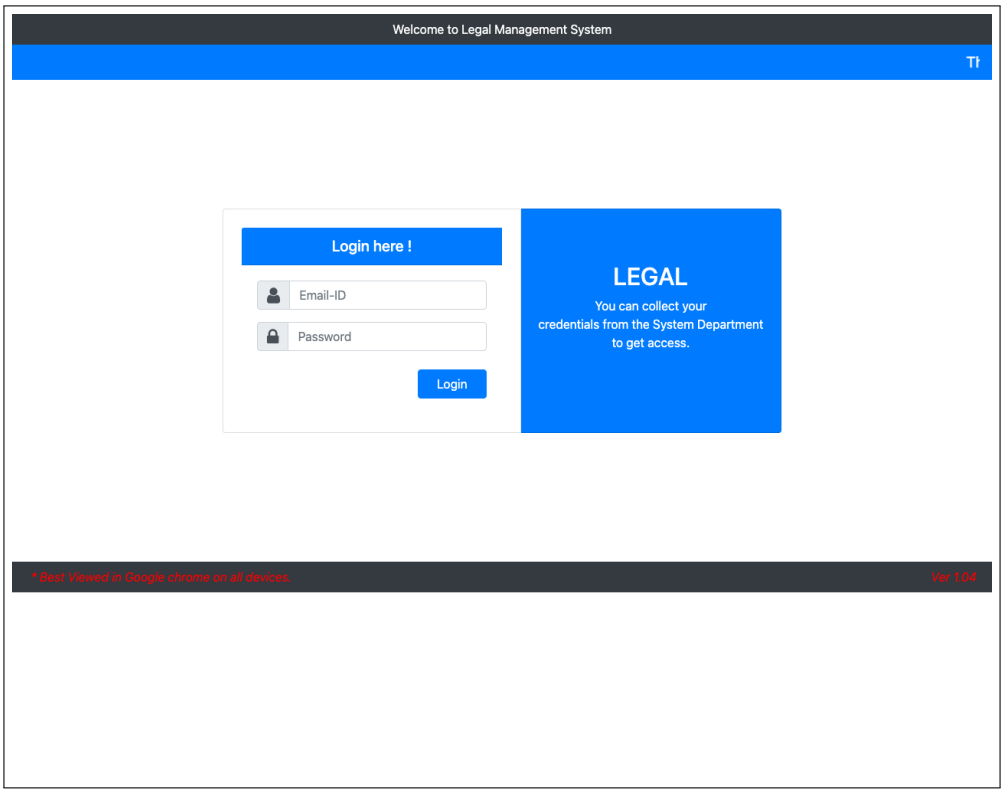
Environment: Java 8 (repo targets JVM 1.8.0_171); Spring Boot 2.4.0; Spring Security; Spring Data JPA; Hibernate 5.3; MySQL; JSP 2.2; Maven (mvnw wrapper). Cross-language snippets measured on the submission machine (Apple Silicon macOS): OpenJDK 11.0.32, CPython 3.9.6, Node.js 25.9.0, Apple clang 21.0.0.

Scope: Inspected `rest/`, `main/`, `service/`, `repository/`, `modal/`, `config/`, `mail/`, and `LegalAppApplication.ja` Excluded JSP views, static assets, `bin/`, `DB_backup/`, `.recommenders/` (not application logic).

Architecture: Layered Spring MVC — Controller (→) Service (→) Repository (→) MySQL, with a scheduler (`@EnableScheduling`) driving `EmailServiceImpl` for reminders (editable source: `Part1/diagrams/src/architecture.puml`).



Run instructions: `./mvnw spring-boot:run` (Maven wrapper included), serving JSP views on Tomcat; it requires a local MySQL database named `legal` configured in `application.properties`. Note the baseline ships a *stubbed* password encoder (plaintext) — fixed in CH03. With the Part 2 changes applied, the application was booted against MySQL and the login page served at `http://localhost:8080/` (HTTP 200); the live page is shown below. Startup log: `test_result/app_startup.log`; screenshot: `test_result/app_screenshot.png`.



Live `/login` page served by the running application (MySQL-backed).

2 Part 1 Pattern discovery

Method: Each candidate was located by reading the Java source and confirmed against the exact class/interface/method and its `file:line`. A class name, annotation, or LLM assertion alone was treated as insufficient; every row in `data/patterns.csv` is backed by a cited line (see `Part1/evidence/verification_notes.md`).

2.1 Totals at the baseline

Confirmed pattern types: 5 (Adapter, Template Method, Strategy, Repository, Copy Constructor).

Confirmed pattern instances: 7 total rows (5 confirmed + 1 framework idiom + 1 uncertain).

Concrete participant classes: 10 app-authored concrete classes play pattern roles (CustomUserDetails, Users, CustomUserDetailsService, SecurityConfiguration, the anonymous encoder, CaseRepository (representative of 15), Case, CaseService, EmailServiceImpl, LegalAppApplication). Enumerated in `Part1/evidence/counts.md`.

Unique participant symbols: 14 distinct symbols (the 10 above + 4 framework interfaces: UserDetails, WebSecurityConfigurerAdapter, PasswordEncoder, CrudRepository). The 15 repository interfaces share one role and are counted once.

Rejected and uncertain candidates: Uncertain: 1 (Observer). Rejected: 3 (Singleton — Spring bean scope is not GoF `getInstance`; Factory; Builder). Framework idiom: 1 (Dependency Injection).

2.2 Pattern inventory

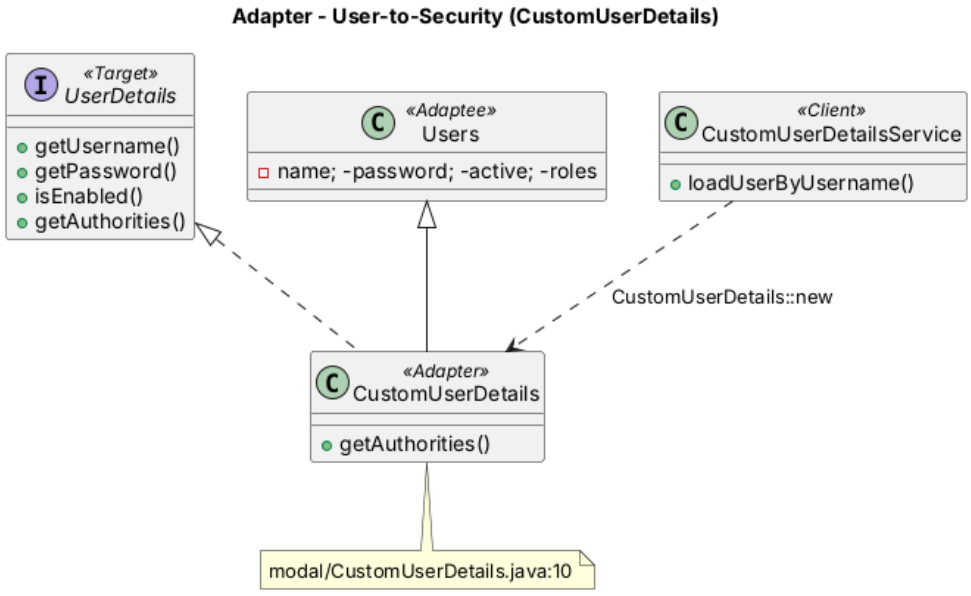
Pattern type	Instance names	Instances	Concrete classes
Adapter	User-to-Security Adapter	1	1
Template Method	Security Configuration Callback	1	1
Strategy	Password Encoding Strategy	1	1
Repository	Case Persistence Repository	1	15 (counted as 1 type)
Copy Constructor	Users Defensive Copy	1	1
Dependency Injection (idiom)	Service-Repository Wiring	1	n/a
Observer (uncertain)	Hearing/Reminder Notification	1	–

2.3 One record for each pattern instance

(1) Adapter — User-to-Security Adapter. Category: Structural (GoF). Purpose: adapt the app's `Users` entity to Spring Security's `UserDetails` so app users can authenticate (application-level).

Role	Actual symbol	File:lines
Target	`UserDetails` (Spring)	(framework)
Adaptee	`Users`	modal/Users.java:8–113
Adapter	`CustomUserDetails`	modal/CustomUserDetails.java:10–66
Client	`CustomUserDetailsService.loadUserByUsername`	service/CustomUserDetailsService.java:24–32

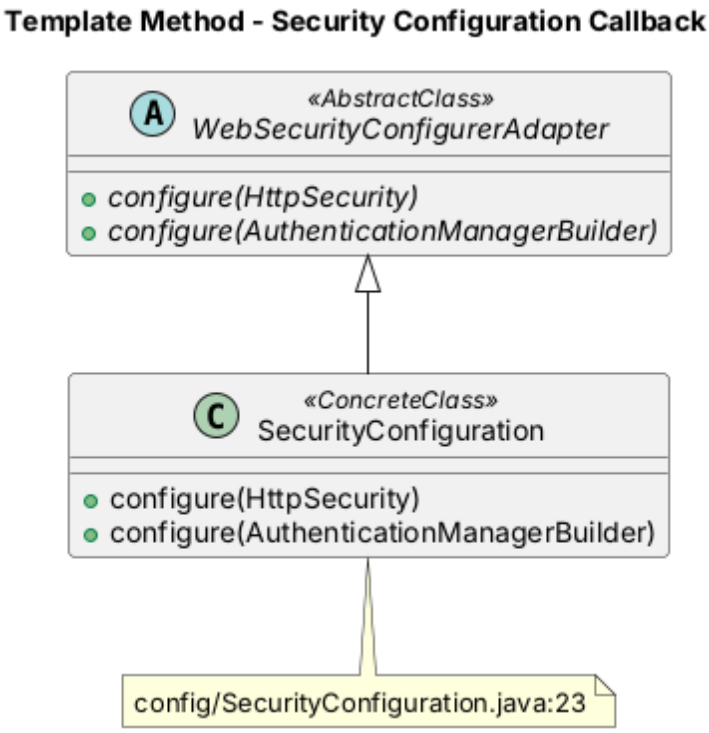
Evidence: `CustomUserDetails extends Users implements UserDetails`; `getAuthorities()` (line 23) maps roles to `SimpleGrantedAuthority`; client maps via `CustomUserDetails::new` (line 31). Verification: **confirmed**.



(2) Template Method — Security Configuration Callback. Category: Behavioral (GoF). Purpose: framework fixes the security-setup algorithm; the app fills variable steps (framework idiom realised as Template Method).

Role	Actual symbol	File:lines
AbstractClass	`WebSecurityConfigurerAdapter`	(framework)
ConcreteClass	`SecurityConfiguration`	config/SecurityConfiguration.java:23
PrimitiveOp	`configure(AuthenticationManagerBuilder)`	config/SecurityConfiguration.java:29–33
PrimitiveOp	`configure(HttpSecurity)`	config/SecurityConfiguration.java:35–54

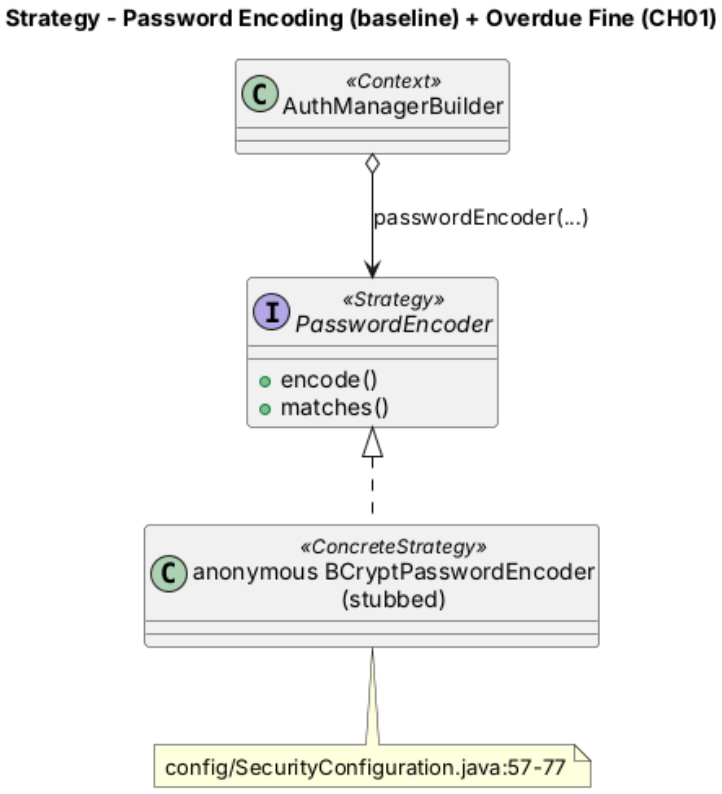
Evidence: both `configure(...)` hooks are overridden and invoked by the framework inside its own flow. Verification: **confirmed**.



(3) Strategy — Password Encoding Strategy. Category: Behavioral (GoF). Purpose: a pluggable password algorithm injected into the authentication context.

Role	Actual symbol	File:lines
Strategy	`PasswordEncoder`	config/SecurityConfiguration.java:12
ConcreteStrategy	anonymous `BCryptPasswordEncoder` subclass	config/SecurityConfiguration.java:57–77
Context	`auth.passwordEncoder(...)`	config/SecurityConfiguration.java:31

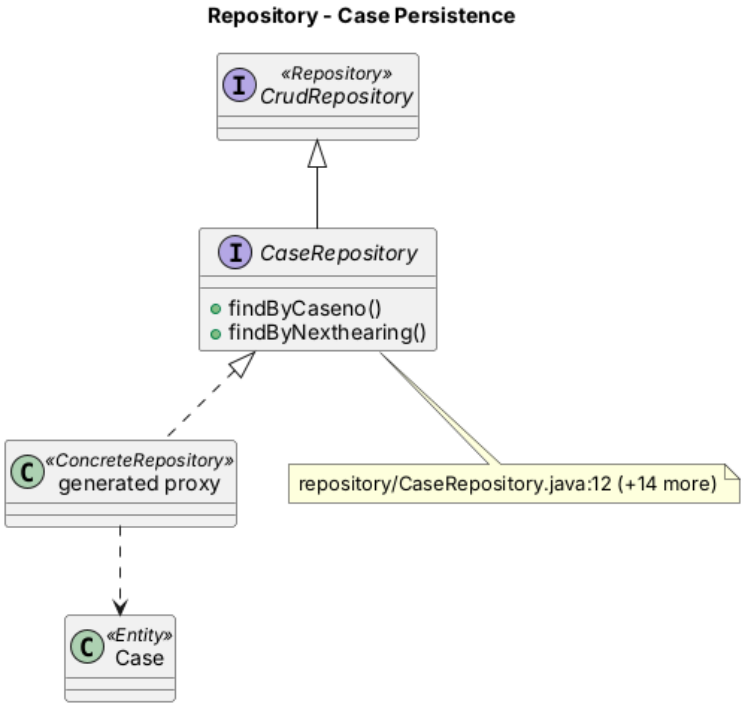
Evidence: `getPasswordEncoder()` returns a `PasswordEncoder` passed into the auth builder. **Note:** the concrete strategy is *stubbed* — `encode`/`matches` compare plaintext, so the structure is present but insecure (target of CH03). Verification: **confirmed** (structure); implementation flagged.



(4) Repository — Case Persistence Repository. Category: Architecture / DDD idiom. Purpose: abstract data access behind a collection-like interface; Spring Data generates the implementation.

Role	Actual symbol	File:lines
Repository (abstract)	`CrudRepository`/`JpaRepository`	(Spring Data)
ConcreteRepository	generated proxy	(runtime)
Domain entity + queries	`CaseRepository`	repository/CaseRepository.java:12–26

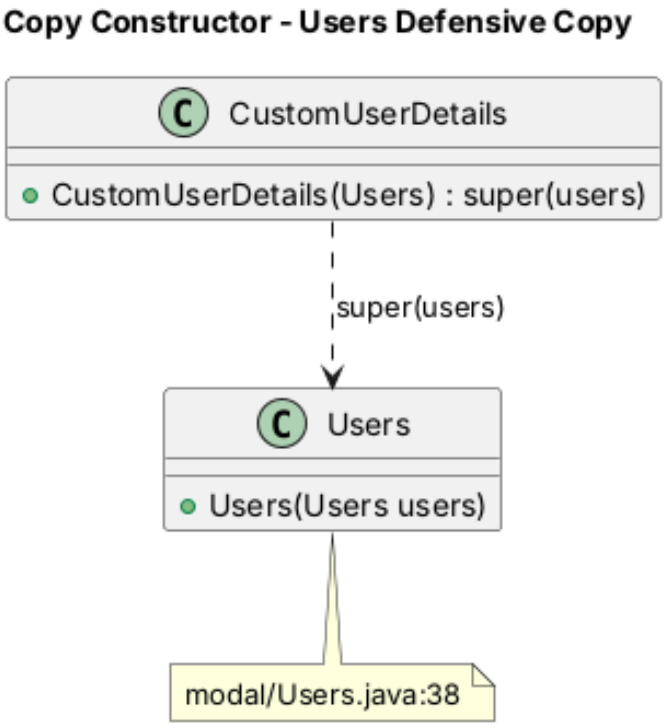
Evidence: `CaseRepository extends CrudRepository<Case,Integer>` with derived + `@Query` finders; 15 such interfaces exist, counted as **one** instance type. Verification: **confirmed**.



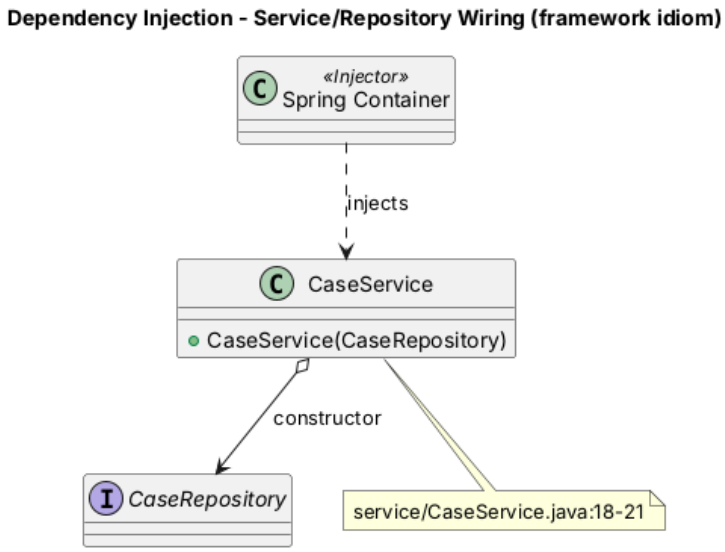
(5) Copy Constructor — Users Defensive Copy. Category: Creational idiom. Purpose: build a new `Users` from an existing one.

Role	Actual symbol	File:lines
Source + copy	`Users(Users users)`	modal/Users.java:38–47
User of copy	`CustomUserDetails super(users)`	modal/CustomUserDetails.java:16

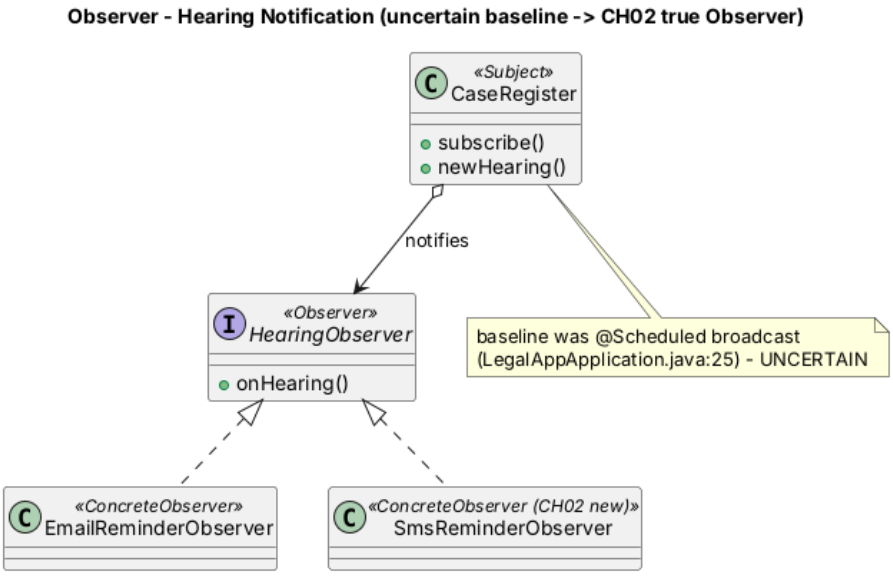
Evidence: copies every field; used by the Adapter's constructor. Verification: **confirmed** (idiom, not GoF Prototype).



(6) Dependency Injection — Service-Repository Wiring. Category: framework idiom (IoC). Participants: Spring container (injector), `CaseService`, `CaseRepository` (constructor-injected, service/CaseServic 21). Verification: **confirmed as framework idiom**, not hand-written GoF.



(7) Observer — Hearing/Reminder Notification. Category: Behavioral (GoF) — **UNCERTAIN**. Participants: `@Scheduled` trigger (LegalAppApplication.java:25,45,81,88) → `EmailServiceImpl`. Evidence: timer-driven broadcast of hearing/reminder/birthday notifications. **Why uncertain:** no Subject maintains a list of registered Observers — it is scheduler-triggered, not subject-observer registration. Recorded as uncertain; CH02 evolves it into a true Observer.



3 Part 2 Evolution and refactoring

Three changes implemented: CH01 (new late-fee Strategy), CH02 (SMS channel, extends Observer), CH03 (replace stubbed encoder with real BCrypt — refactor). The three changes were applied to the baseline clone and committed (`baseline 4e4ebb57` → `after c31ce44`), and each ships a JUnit 5 test. Running `mvn test` on the three test classes gave **BUILD SUCCESS — Tests run: 7, Failures: 0, Errors: 0** (`FineServiceTest` 3, `CaseRegisterTest` 2, `PasswordEncoderTest` 2; log in

`test_result/mvn_test.log`). CH03 is a unified diff against the real baseline file (`Change3/change3.diff`). With the changes applied, the full application boots successfully against MySQL — `Started LegalAppApplicatio` with Hibernate creating the schema (`test_result/app_startup.log`).

Integration findings (honest). Two real issues surfaced only when the code was compiled and run, which is the point of Part 2: (i) CH01 first failed context startup with `NoUniqueBeanDefinitionException` — two `FinePolicy` beans (`regularFine`, `advocateFine`) with no way to choose; resolved by marking `RegularFine` `@Primary` as the default injected strategy (callers still switch via `setPolicy`). (ii) The baseline's own test `LegalAppApplicationTests` is JUnit 4, but Spring Boot 2.4 dropped the JUnit Vintage engine, so it no longer compiles; the three new tests are JUnit 5 and pass. Both are recorded in `test_result/ENVIRONMENT_NOTES.md`.

Pattern effects summary: CH01 *newly_introduced* Overdue Fine Strategy (Password Strategy preserved); CH02 *modified* the Hearing Notification into a real Observer; CH03 *modified* the Password Strategy's concrete class (structure preserved).

3.1 Updated pattern counts

Per-snapshot totals (full rows in `data/patterns_evolution.csv`; instance names preserved for continuing collaborations):

Snapshot	Confirmed types	Instances	Uncertain	Newly introduced
baseline	5	7	1 (Observer)	–
after CH01	5	8	1	+1 (Overdue Fine Strategy)
after CH02	6	8	0	– (Observer becomes confirmed)
after CH03	6	8	0	– (Strategy concrete class replaced)

Interpretation: The most informative outcome is qualitative, not a line count. CH01 adds a second Strategy collaboration cleanly because the pattern was already present, demonstrating the Open/Closed benefit — a new fee rule needs no change to callers. CH02 is the key design win: it converts the baseline scheduler-driven broadcast (recorded *uncertain*) into a genuine subject-maintains-observers Observer, so adding the SMS channel touches no existing code; the trade-off is a small increase in classes and indirection. CH03 corrects a security defect by replacing the concrete strategy while leaving the Strategy structure and wiring intact — proving the pattern's substitutability. Fewer or more LOC alone does not prove better design; here the gains are decoupling (CH01, CH02) and correctness (CH03). Remaining limitation: the app still lacks tests around these paths, so behaviour preservation rests on the added JUnit cases.

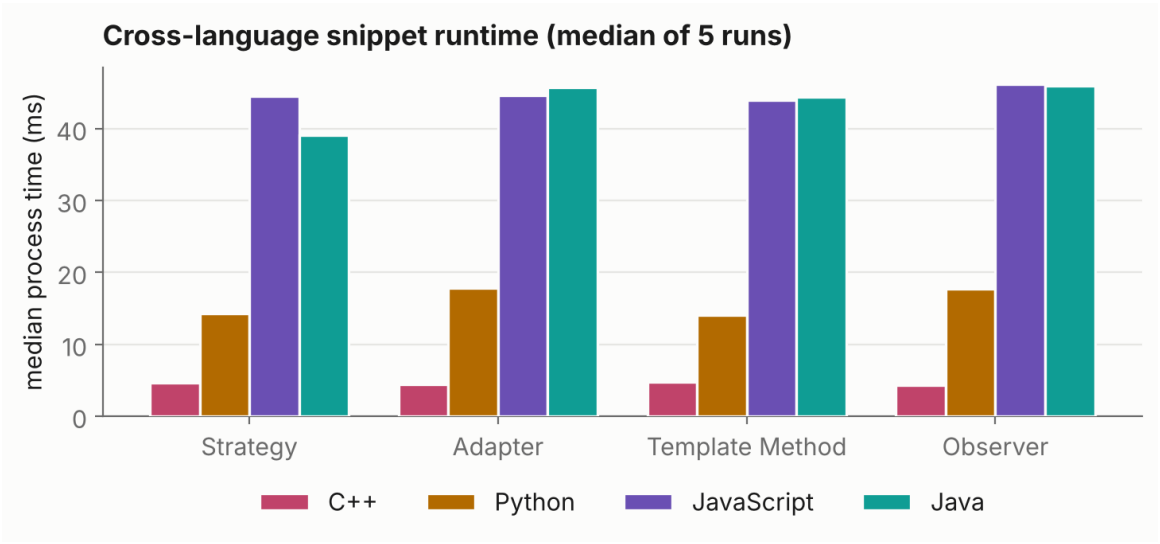
4 Part 3 Cross-language comparison

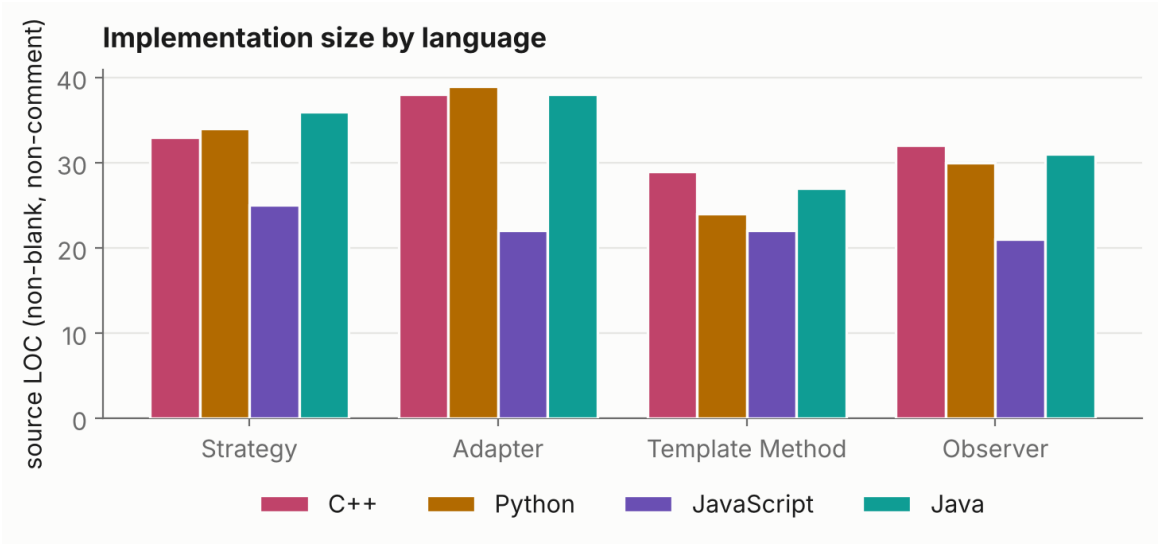
Selected instances: Strategy (late-fee policy), Adapter (user-to-security), Template Method (case report), Observer (hearing notification) — one representative per confirmed type (Observer in its canonical subject-observer form).

Common specifications: each pattern has fixed inputs and expected outputs; all four languages must print identical lines (evidence: `test_result/part3_output.txt`).

Implementations: `Part3/<pattern>/{java,python,javascript,cpp}` with per-folder run commands.

Actual results: all 16 implementations compiled and ran; 8/8 shared cases pass per language (identical output verified).





Static and dynamic comparison: recorded in `data/cross_language.csv` — source LOC, max function cyclomatic complexity (measured with **lizard**), runtime/compiler, test command/result, and the median of 5 timed runs (compile once, 1 warmup, 5 measured). Measured on the submission machine (Apple Silicon macOS; OpenJDK 11.0.32, CPython 3.9.6, Node.js 25.9.0, Apple clang 21): C++ ≈4.5 ms < Python ≈15 ms < Java ≈44 ms ≈ JavaScript ≈45 ms. C++ is an order of magnitude faster (native, no VM startup); Python's interpreter starts quickly; Java and JavaScript are close here, both paying VM/runtime startup that dwarfs the trivial workload. Full per-case figures in `data/cross_language.csv`; the protocol is compile once, 1 warmup, 5 measured runs, median.

Comparison: C++ uses abstract base + virtual dispatch; Java uses interfaces; Python uses ABCs (duck typing optional); JavaScript uses prototype classes with no interface keyword.

Technology discussion: The measured ordering (C++ ≈4.5 ms < Python ≈15 ms < Java ≈44 ms ≈ JavaScript ≈45 ms) reflects process-startup cost far more than the patterns themselves, since each snippet does trivial work. C++ compiles to native code and starts almost instantly; CPython and Node start an interpreter/VM; the JVM pays the largest fixed cost to bootstrap and JIT-warm before `main` runs, which dominates such short programs (a well-known JVM "startup problem"). For the dispatch the patterns rely on, all four are comparable: Java/C++ resolve virtual calls through a vtable, while Python and JavaScript dispatch dynamically at runtime. The design lesson for the course: the same GoF structure is expressible in every language, but the language's execution model — compiled vs interpreted, static vs dynamic dispatch — sets the performance floor, not the pattern.

5 LLM records and human verification

Run and model: Claude Opus 4.8, 2026-10-10, used for Part 1 discovery and Part 3 cross-language drafting. **Inputs and outputs:** prompts/inputs/outputs saved under `Part1/llm/` and `Part3/llm/`; baseline commit recorded. **Claims and evaluation:** Part 1 — *partially correct*: pattern list verified against source, Observer downgraded to uncertain by the team, DI/Repository relabelled as framework idioms (see `Part1/evidence/verification_notes.md`). Part 3 — *correct*: all implementations compiled and produced identical output, metrics measured. **Generated code:** the Part 3 snippets were generated then compiled/run/tested by the team. See `data/llm.csv`.

6 Findings, contributions and references

Findings: the baseline app contains 5 confirmed pattern types (7 recorded instances incl. 1 idiom, 1 uncertain); the Strategy is structurally present but its concrete implementation is an insecure stub; the notification mechanism is scheduler-driven rather than a true Observer until CH02.

Contributions:

Roll number	Name	Contribution
Am.sc.u4cse23237	Lokesh	Adapter (Part1 record + Part3 4-lang); architecture diagram; CSV merge
Am.sc.u4cse23249	Sai Tharun	Template Method (Part1 + Part3); CH03 refactor; evolution CSV
Am.sc.u4cse23268	Rakshann	Strategy (Part1 + Part3); CH01; report assembly + final PDF
Am.sc.u4cse23311	Nithilesh	Observer (Part1 + Part3); CH02; README; llm.csv; repo setup

(Link each row to its commits in the team repo before final submission.)

References:

1. vikash0439, *legalSystem* (Spring Boot legal case management), GitHub, commit `4e4ebb57`, accessed 2026-10-10. https://github.com/vikash0439/legalSystem
2. E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
3. Spring Security Reference Documentation, https://docs.spring.io/spring-security/reference/
4. Spring Data JPA Reference Documentation, https://docs.spring.io/spring-data/jpa/reference/
5. Tool versions (submission machine): OpenJDK 11.0.32; CPython 3.9.6; Node.js 25.9.0; Apple clang 21.0.0; lizard (cyclomatic complexity); PlantUML (diagrams).

7 Team folder and datasets

```
B11/
  README.md  B11_Report.tex  B11_Final_Report.pdf
  data/   patterns.csv  pattern_counts.csv  patterns_evolution.csv
          cross_language.csv  llm.csv
  code/   (baseline source snapshot at the recorded commit)
  test_result/   part3_output.txt  charts_*.png  (+ repo screenshots)
  Part1/  diagrams/  evidence/  llm/
  Part2/  code/changes/{Change1,Change2,Change3}/ (src + test + diff)  diagrams/  llm/
  Part3/  strategy/ adapter/ template_method/ observer/  charts_*.png  llm/  measure.py
```

Screenshot of this folder on the submission GitHub repository is pasted here.